

Python Fast API

1. What is FastAPI?

FastAPI = modern Python framework for building APIs

- **Framework** = provides reusable web components
 - **API** = exposes application functionality
 - **Fast** = built on ASGI (asynchronous server interface)
 - **Validation** = uses Python type hints
 - **Documentation** = automatically generates OpenAPI docs
-

2. Why use FastAPI?

FastAPI = API development with performance + validation + documentation

- **Performance** = supports asynchronous requests
 - **Validation** = validates request data automatically
 - **Type hints** = define expected data types
 - **Documentation** = generates Swagger UI automatically
 - **Developer experience** = less boilerplate code
-

3. What is ASGI?

ASGI = interface for asynchronous Python web applications

- **Interface** = standard between server and application
 - **Async** = supports non-blocking operations
 - **WebSocket** = supports persistent connections
 - **FastAPI** = built for ASGI
 - **Server** = commonly Uvicorn
-

4. What is Uvicorn?

Uvicorn = ASGI server that runs FastAPI

- **Server** = accepts HTTP requests
 - **ASGI** = communicates with FastAPI application
 - **Event loop** = manages asynchronous operations
 - **Example** = `uvicorn main:app --reload`
-

5. How do you create a basic FastAPI application?

FastAPI app = application instance + route

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
def home():
    return {"message": "Hello"}
```

- **FastAPI()** = creates application
 - **@app.get()** = defines GET route
 - **return** = automatically converted to JSON response
-

6. What is a route in FastAPI?

Route = URL + HTTP method mapped to function

- **URL** = `/users`
 - **Method** = GET
 - **Handler** = Python function
 - **Example** = `@app.get("/users")`
-

7. How does FastAPI handle path parameters?

Path parameter = value extracted from URL

```
@app.get("/users/{user_id}")
def get_user(user_id: int):
    return {"id": user_id}
```

- `{user_id}` = path parameter
 - `int` = expected data type
 - **Validation** = FastAPI validates automatically
-

8. How does FastAPI handle query parameters?

Query parameter = optional/customizable URL input

```
@app.get("/users")
def get_users(page: int = 1, limit: int = 20):
    ...
```

- `page` = query parameter
 - `limit` = query parameter
 - **Default** = used when value isn't provided
 - **URL** = `/users?page=2&limit=20`
-

9. Path parameter vs query parameter?

Path = identify resource, query = customize request

- **Path** = `/users/123`
 - **Query** = `/users?role=admin`
 - **Path** = usually required
 - **Query** = usually optional
-

10. How does FastAPI handle request bodies?

Request body = data sent to API

```
from pydantic import BaseModel

class User(BaseModel):
    name: str
    age: int

@app.post("/users")
def create_user(user: User):
    return user
```

- **Pydantic** = data validation library
 - **User** = request schema
 - **Validation** = automatic
 - **Invalid data** = FastAPI returns validation error
-

11. What is Pydantic?

Pydantic = library for data validation using Python types

- **Model** = defines expected structure
 - **Type hints** = define field types
 - **Validation** = checks incoming data
 - **Serialization** = converts data into standard formats
-

12. What is a Pydantic model?

Pydantic model = schema for structured data

```
class User(BaseModel):
    name: str
    age: int
```

- **Schema** = expected data structure
 - **Field** = individual property
 - **Type** = expected data type
 - **Validation** = automatically performed
-

13. What is response_model?

response_model = defines and validates API response structure

```
@app.get("/users/{id}", response_model=User)
def get_user(id: int):
    ...
```

- **Input model** = validates request
 - **Response model** = validates response
 - **Serialization** = controls returned fields
 - **Benefit** = consistent API contract
-

14. How does FastAPI validate requests?

FastAPI = validates input using Python type hints + Pydantic

- **Path** = validates path parameters
 - **Query** = validates query parameters
 - **Body** = validates Pydantic models
 - **Invalid input** = returns `422` validation response
-

15. What is dependency injection in FastAPI?

Dependency Injection = provide required functionality to route

```
@app.get("/users")
def get_users(db = Depends(get_db)):
```

• • •

- **Depends** = declares dependency
 - **Dependency** = reusable functionality
 - **Example** = database connection
 - **Example** = authentication
 - **Benefit** = reusable and testable code
-

16. Why use dependency injection?

Dependency Injection = avoid repeating common logic

- **Authentication** = verify user
 - **Database** = provide DB session
 - **Permissions** = check access
 - **Configuration** = provide settings
 - **Benefit** = separation of concerns (each component has one responsibility)
-

17. What is middleware in FastAPI?

Middleware = code executed around every request

- **Before request** = execute logic
 - **Request** = reaches endpoint
 - **After response** = execute logic
 - **Use cases** = logging, CORS, timing, authentication
-

18. Middleware vs dependency?

Middleware = global request processing, dependency = route-level reusable logic

- **Middleware** = applies broadly
- **Dependency** = applies to selected routes

- **Middleware** = request/response lifecycle
 - **Dependency** = provides values or checks
-

19. What is CORS?

CORS = controls cross-origin browser requests

- **Origin** = protocol + domain + port
 - **Example** = React frontend → FastAPI backend
 - **Purpose** = browser security
 - **FastAPI** = provides CORS middleware
-

20. How do you handle exceptions in FastAPI?

Exception handling = return controlled error responses

```
from fastapi import HTTPException

raise HTTPException(
    status_code=404,
    detail="User not found"
)
```

- **HTTPException** = FastAPI HTTP error
 - **Status code** = communicates failure type
 - **Detail** = error information
 - **Production** = use consistent error format
-

21. How do you structure a FastAPI project?

FastAPI structure = separate API, business logic, and data access

```
app/
├─ main.py
├─ routers/
```

```
|— services/  
|— models/  
|— schemas/  
|— repositories/  
|— dependencies/  
└— config/
```

- **Routers** = API endpoints
 - **Services** = business logic
 - **Repositories** = database operations
 - **Schemas** = request/response models
 - **Config** = application configuration
-

22. Router vs service?

Router = HTTP layer, service = business layer

- **Router** = receives HTTP request
 - **Router** = validates/returns response
 - **Service** = business logic
 - **Service** = should not depend heavily on HTTP details
-

23. What is async/await in FastAPI?

async/await = non-blocking asynchronous execution

- **async** = defines asynchronous function
 - **await** = waits without blocking event loop
 - **Good for** = I/O-bound operations
 - **Examples** = HTTP calls, async database queries
-

24. When should you use async in FastAPI?

Use async = when underlying operations are asynchronous

- **Async DB driver** = use `async`

- **Async HTTP client** = use `async`
 - **CPU-heavy task** = don't expect `async` to make it faster
 - **Blocking library** = can block event loop
-

25. What is the event loop?

Event loop = manages asynchronous tasks

- **Task** = asynchronous operation
 - **Waiting** = event loop handles other tasks
 - **Benefit** = efficient I/O concurrency
 - **Important** = blocking code can stop the loop
-

26. Async vs sync endpoint?

Async = non-blocking I/O, sync = traditional blocking execution

- `async def` = suitable for async libraries
 - `def` = suitable for blocking/synchronous code
 - **Async** = doesn't automatically make CPU work faster
 - **Choice** = depends on dependencies being used
-

27. How does FastAPI handle authentication?

Authentication = verify client identity

- **JWT** = signed authentication token
 - **OAuth2** = authorization framework
 - **Bearer token** = token sent in Authorization header
 - **Dependency** = commonly used for authentication checks
-

28. How do you connect FastAPI to a database?

Database integration = driver/ORM + dependency-managed connections

- **Driver** = communicates with database
 - **ORM** = maps objects to database records
 - **Session** = database interaction context
 - **Dependency** = manages session lifecycle
 - **Examples** = SQLAlchemy, SQLModel, asyncpg
-

29. What is SQLAlchemy?

SQLAlchemy = Python SQL toolkit and ORM

- **ORM** = maps Python objects to database tables
 - **Engine** = manages database connectivity
 - **Session** = manages database operations
 - **Query** = retrieves or modifies data
-

30. How do you manage database sessions?

DB session = create → use → close

- **Create** = obtain session
 - **Use** = execute queries
 - **Commit** = persist changes
 - **Rollback** = undo failed transaction
 - **Close** = release resources
-

31. How do you handle background tasks?

Background task = execute work after sending response

- **Use** = lightweight post-response work
- **Example** = logging, notifications
- **FastAPI** = provides `BackgroundTasks`
- **Heavy jobs** = use Celery (distributed task queue) or similar worker system

32. How do you handle long-running tasks?

Long-running task = move work outside API request lifecycle

```
Client
  ↓
FastAPI
  ↓
Queue
  ↓
Worker
  ↓
Database / External Service
```

- **API** = accepts request
- **Queue** = stores job
- **Worker** = processes job
- **Response** = returns job ID
- **Client** = checks status later

33. How do you improve FastAPI performance?

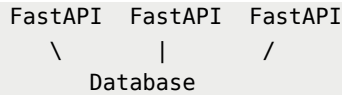
Performance = efficient I/O + database + architecture

- **Async I/O** = handle concurrent I/O efficiently
- **Connection pool** = reuse database connections
- **Caching** = reduce repeated queries
- **Pagination** = control response size
- **Workers** = use multiple processes when appropriate

34. How do you scale FastAPI?

Scaling = multiple application instances behind a load balancer

```
Load Balancer
 /   |   \
```



- **Horizontal scaling** = add more instances
- **Load balancer** = distributes requests
- **Stateless API** = allows flexible scaling
- **Shared state** = move to external systems like Redis/database

35. How do you test FastAPI APIs?

API testing = verify endpoints independently

- **Unit test** = test individual logic
- **Integration test** = test component interaction
- **TestClient** = simulate API requests
- **Mock** = replace external dependencies
- **Pytest** = common Python testing framework

36. What is OpenAPI in FastAPI?

OpenAPI = machine-readable API specification

- **Defines** = endpoints
- **Defines** = request/response schemas
- **Defines** = parameters and status codes
- **FastAPI** = generates OpenAPI automatically
- **Swagger UI** = interactive API documentation

37. What is Swagger UI?

Swagger UI = interactive documentation for APIs

- **Generated from** = OpenAPI specification

- **Shows** = endpoints and schemas
 - **Allows** = directly testing APIs
 - **FastAPI** = provides it automatically
-

38. Flask vs FastAPI?

Flask = minimal web framework, FastAPI = API-focused modern framework

- **Flask** = lightweight and flexible
 - **FastAPI** = built-in validation and OpenAPI
 - **Flask** = synchronous by default
 - **FastAPI** = strong async support
 - **Choice** = depends on application requirements and ecosystem
-

39. FastAPI vs Django?

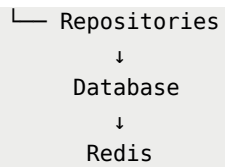
FastAPI = API-focused, Django = full web framework

- **FastAPI** = lightweight API services
 - **Django** = batteries-included (many built-in features)
 - **Django** = ORM, admin, authentication ecosystem
 - **FastAPI** = more modular
 - **Choice** = depends on system requirements
-

40. How would you design a production FastAPI service?

Production FastAPI = API + business logic + database + infrastructure

```
Client
  ↓
Load Balancer
  ↓
FastAPI
  ├── Authentication
  ├── Routers
  └── Services
```



- **Load Balancer** = distributes traffic
- **FastAPI** = HTTP/API layer
- **Services** = business logic
- **Repositories** = database access
- **Redis** = caching/session/rate limiting
- **Observability** = logs, metrics, tracing (request-flow tracking)